R1.01 - Initiation au Développement : Variables, Mémoire RAM, Structures Conditionnelles et Répétitives

Thème principal : Bases fondamentales de l'algorithmique et de la programmation impérative

Synthèse exécutive : Ce cours pose les fondations absolues du développement informatique pour le BUT Informatique. Il explique la manière dont un ordinateur manipule la mémoire vive (RAM) à travers des variables typées et l'opération d'affectation destructive. Il détaille la dérivation du flux d'exécution grâce aux structures conditionnelles (SI / ALORS / SINON) fondées sur la logique booléenne, ainsi que l'automatisation des traitements répétitifs via les boucles (POUR et TANT QUE). L'objectif pédagogique est d'apprendre à formaliser un problème métier sous forme d'un algorithme rigoureux, optimisé et exempt de bugs critiques comme les boucles infinies.

1. Les Variables, le Typage et la Représentation en Mémoire RAM

En programmation impérative, le processeur exécute des instructions en manipulant des données conservées temporairement dans la mémoire vive (RAM). Pour éviter au développeur de manipuler des adresses hexadécimales brutes et complexes (ex: 0x7FFF5FBFF058), l'algorithmique introduit l'abstraction de la variable. Toute variable doit obligatoirement être déclarée avant son utilisation. La déclaration indique au compilateur ou à l'interprète le nom de la variable (son identificateur) et son type. Le type détermine l'espace physique réservé en mémoire vive. Par exemple, un entier standard requiert généralement 32 bits (4 octets), tandis qu'un nombre réel à virgule flottante requiert 64 bits (8 octets). L'opération d'affectation respecte un ordre d'évaluation strict en deux phases : d'abord, l'expression située à droite de l'opérateur est intégralement calculée dans les registres du processeur ; ensuite, le résultat est écrit dans la case mémoire associée à la variable située à gauche. Cette écriture est dite 'destructive' car l'ancienne valeur présente dans le tiroir mémoire est définitivement effacée et ne peut plus être récupérée.

- Variable : Emplacement mémoire nommé (adresse abstraite en RAM) permettant de conserver et de modifier une valeur pendant l'exécution d'un programme.
- Typage : Spécification du format de donnée (Entier, Réel, Chaîne de caractères, Booléen) fixant sa taille d'allocation en octets (ex: 4 octets pour un entier, 8 octets pour un réel double précision) et les opérations autorisées.
- Affectation (notée <- ou =) : Opération séquentielle et destructive consistant à évaluer l'expression située à droite du signe, puis à stocker ce résultat dans la variable située à gauche, écrasant irréversiblement son ancien contenu.

Cas concret enseignant : *Le professeur compare la mémoire RAM à un grand meuble à tiroirs : déclarer la variable 'age : Entier', c'est coller une étiquette 'age' sur un tiroir spécifique et fabriquer ce tiroir à la dimension exacte pour recevoir un nombre entier. Placer une nouvelle valeur dans le tiroir chasse et détruit instantanément ce qui s'y trouvait auparavant.*

2. Les Structures Conditionnelles et l'Algèbre Booléenne

Par défaut, un programme informatique s'exécute de façon séquentielle, ligne par ligne. Les structures conditionnelles rompent cette linéarité en permettant à l'algorithme d'emprunter des chemins d'exécution différents selon l'état des variables ou les saisies faites par l'utilisateur. Le processeur évalue d'abord l'expression booléenne. Si la condition renvoie VRAI, les instructions du bloc 'ALORS' sont exécutées et le bloc 'SINON' est totalement ignoré. Si la condition renvoie FAUX, le processeur saute directement au bloc 'SINON' (lorsqu'il existe) ou poursuit à l'instruction suivant la structure conditionnelle. Lors de la conception d'un algorithme, il est fondamental de garantir l'exhaustivité des cas. Une erreur classique en examen consiste à oublier de traiter des cas limites (ex: valeurs négatives, égalités exactes), ce qui entraîne des comportements indéterminés de l'application.

- Condition Booléenne : Expression logique dont l'évaluation produit exclusivement la valeur VRAI (True) ou FAUX (False).
- Bloc SI / ALORS / SINON : Structure de contrôle permettant de dériver l'exécution du code selon le résultat d'un test booléen.

• Opérateurs Logiques (ET, OU, NON) : Connecteurs permettant de combiner plusieurs conditions simples pour former des règles de décision complexes.

Cas concret enseignant : *Exemple du contrôle d'accès au parking du campus de Bayonne/Anglet : Si (carte_etudiant_valide == VRAI ET parking_complet == FAUX) ALORS lever la barrière ; SINON afficher le message d'erreur 'Accès Refusé' sur l'écran.*

3. Les Structures Répétitives (Boucles POUR et TANT QUE)

Les boucles évitent la duplication de code source en répétant un même ensemble d'instructions. Le choix du type de boucle est une décision d'architecture algorithmique fondamentale : 1. La boucle POUR est préconisée lorsque le nombre de répétitions est déterminé au départ (par exemple, parcourir les 500 étudiants d'une promotion). La variable de contrôle (le compteur) s'incrémente automatiquement à chaque itération. 2. La boucle TANT QUE est utilisée lorsque la répétition dépend d'un événement dynamique ou d'une saisie utilisateur (par exemple, demander un mot de passe jusqu'à ce qu'il soit correct). La condition est évaluée AVANT chaque itération. Si la condition est fausse dès le départ, le corps de la boucle n'est jamais exécuté (0 itération). Règle d'or absolue d'ingénierie : Pour éviter une boucle infinie dans un TANT QUE, au moins une variable engagée dans la condition de test doit obligatoirement être modifiée à l'intérieur du corps de la boucle.

• Boucle POUR (FOR) : Structure itérative déterministe avec compteur automatique, réservée aux cas où le nombre d'itérations est fixe et connu avant le début de la boucle.

• Boucle TANT QUE (WHILE) : Structure itérative conditionnelle, exécutée tant qu'une condition logique reste vraie, adaptée aux répétitions indéterminées.

• Boucle Infinie : Dysfonctionnement majeur survenant lorsque la condition de sortie d'une boucle TANT QUE ne devient jamais fausse, bloquant indéfiniment le programme.

Cas concret enseignant : *Le bug du distributeur automatique de billets (DAB) : si l'algorithme oublie de remettre la variable 'carte_inseree' à FAUX après l'éjection physique de la carte de la fente, la boucle TANT QUE demandant la saisie du code PIN tourne indéfiniment et bloque la machine.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi doit-on obligatoirement spécifier le type d'une variable dans des langages comme C ou C++ alors que ce n'est pas nécessaire en Python ?
Réponse du professeur : Dans les langages à typage statique comme C ou C++, le compilateur a besoin de connaître le type dès la compilation afin de reserver exactement le bon nombre d'octets en mémoire RAM (ex: 4 octets pour un int) et de vérifier la sécurité du code avant l'exécution. En Python, le typage est dynamique : le type est attaché à l'objet stocké en mémoire pendant l'exécution. Cela offre plus de flexibilité d'écriture, mais consomme plus de mémoire et empêche la détection de certaines erreurs avant le lancement du programme.

Question étudiant : Quelle est la différence exacte entre l'opérateur '=' et l'opérateur '==' ?
Réponse du professeur : L'opérateur '=' (ou '<-' en pseudo-code) est une AFFECTATION. C'est une action d'écriture : il prend la valeur calculée à droite et l'enregistre dans la variable à gauche en écrasant sa valeur précédente. L'opérateur '==' est un TEST D'ÉGALITÉ. C'est une consultation : il compare la valeur de gauche et la valeur de droite sans les modifier, et renvoie un booléen (VRAI ou FAUX).

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme complet en pseudo-code qui demande à l'utilisateur de saisir 5 notes (sur 20), calcule la moyenne arithmétique de ces notes, puis affiche 'Admis' si la moyenne est supérieure ou égale à 10.0, ou 'Ajourné' dans le cas contraire.

Résolution pas à pas :
1. Déclaration et initialisation des variables : - somme : Réel <- 0.0 (accumulateur initialisé à 0) - note : Réel (stocke la saisie courante) - moyenne : Réel (stocke le résultat final) - i : Entier (compteur de la boucle) 2. Saisie des notes et accumulation (Le nombre de notes est connu = 5 -> Boucle POUR) : POUR i ALLANT DE 1 À 5 FAIRE Afficher

"Entrez la note numéro ", i, " : " Saisir note somme <- somme + note FIN POUR 3. Calcul de la moyenne arithmétique : moyenne <- somme / 5.0 4. Test conditionnel et affichage du statut : SI moyenne >= 10.0 ALORS Afficher "Résultat : Admis (Moyenne = ", moyenne, ")" SINON Afficher "Résultat : Ajourné (Moyenne = ", moyenne, ")" FIN SI

Résultat : L'algorithme s'exécute de manière déterministe en exactement 5 itérations. La variable 'somme' cumule correctement chaque note saisie sans réinitialisation intempestive dans la boucle. La moyenne réelle est calculée puis testée rigoureusement par la structure conditionnelle pour afficher la décision finale.

■ Auto-Évaluation Flash d'Examen

Q : Que se passe-t-il si la condition d'une boucle TANT QUE est évaluée à FAUX dès le premier contrôle ?

R : Le corps de la boucle n'est jamais exécuté (0 itération). Le programme saute directement à la première instruction située après la structure TANT QUE.

Q : Quelle est la règle stricte pour choisir entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR s'impose lorsque le nombre d'itérations est fixe et connu au départ. La boucle TANT QUE s'impose lorsque le nombre de répétitions dépend d'une condition dynamique évoluant durant l'exécution.

Q : Pourquoi est-il dangereux de comparer deux nombres réels (flottants) directement avec l'opérateur d'égalité '==' ?

R : À cause des imprécisions d'arrondi liées à la représentation binaire des nombres à virgule flottante (norme IEEE 754). Deux calculs mathématiquement égaux peuvent différer d'une quantité minime en binaire. Il faut comparer si la valeur absolue de leur différence est inférieure à un seuil de tolérance (epsilon) : abs(a - b) < 0.00001.